

Wire without phone

Descriere

Clienții unui serviciu de comunicație secret au la dispoziție din partea managerului rețelei un API (application programming interface) uniform, prin care se poate programa un protocol peste canalul de comunicație dat.

Fiecare client, când este înregistrat la început în rețea, primește un număr de identificare, de la 0 la $n-1$, pe care îl știe doar managerul rețelei.

API-ul permite transmiterea de informație într-un singur sens. Cu toate astea, Florin a găsit o modalitate de a facilita și comunicația în sens invers, dar a dispărut în mod misterios.

Cerință

Se alege un emițător și un receptor de la capete opuse firului. Emițătorul vrea să îi transmită receptorului un număr într-un anumit interval.

Scopul problemei este să rezolvați problema pentru cazul când emițătorul este clientul $n-1$ și receptorul este clientul 0. Va trebui să facilitați transmiterea numerelor de la 1 la $2^{32} - 1$.

Florin, expert în securitate, a găsit o soluție prin care se poate transmite informație în sens invers, dar nu a apucat să termine de scris codul. A rămas de scris funcția `step3()`, restul codului fiind corect.

Pentru verificare, veți avea de încărcat **doar** fișierul `client.cpp`. Acesta se va verifica alături de `manager.cpp` și `client.h`, care sunt la fel cu cele pe care le primiți.

Detalii de implementare

Transmiterea de informații

În rețea clientul i poate să transmită *doar* numere naturale *doar* clientului $i+1$, prin funcția `send()`.

Ca să se poată stabili un protocol peste acele numere, fiecare client primește și o unitate de stocare, numită registru. Spre deosebire de numărul de identificare, clienții își pot citi unitatea de stocare prin funcțiile `get_register()` și `set_register()`. Deci dacă oricare client ar socoti potrivit să își scrie în registru orice valoare, poate face asta cu singura restricție să fie între 0 și `MAX_SIZE`, inclusiv.

În acest sens, `send(x)`, adaugă x la registru destinatarului.

Mai jos avem ilustrat un exemplu în care clienții fac acțiuni aleatoare, cu scop demonstrativ.

sequenceDiagram

```
participant A as Ana <br/> #35;0
participant B as Bogdan <br/> #35;1
participant C as Cristi <br/> #35;2
participant M as Manager
autonumber
```

Note over A,C: Toate registrele sunt 0.
 MAX_SIZE = 500

A->>B: send(71)

B->>A: OK

```

activate B
note right of B: registru(Bogdan) += 71
deactivate B

B--xC: send(501)
C->>B: ERR

B->>B: set_register(499)

C--xM: send(0)
M->>C: ERR

A--xB: send(2)
activate B
note right of B: 2 + registru(Bogdan) > MAX_SIZE
deactivate B
B->>A: ERR

A->>B: send(1)
activate B
note right of B: 1 + registru(Bogdan) == MAX_SIZE <br/> registru(Bogdan) = 500
deactivate B
B->>A: OK

A->>B: send(0)
activate B
note right of B: 0 + registru(Bogdan) == MAX_SIZE
deactivate B
B->>A: OK

```

Descrierea pașilor din diagrama de mai sus:

1. Clientul #0 trimite 71 clientului #1.
2. Clientul #1 are registrul 0, așa că poate adăuga 71 cu succes.
3. Clientul #1 trimite 501 clientului #2.
4. Clientul #2 are registrul 0, dar dacă s-ar adăuga 501, ar depăși MAX_SIZE. Operația nu se execută și se întoarce ERR. Valoarea din registrul clientului #2 rămâne 0.
5. Clientul #1 își setează propriul registru cu valoarea 499. Valoarea anterioară e suprascrisă.
6. Clientul #2 trimite 0 la dreapta lui.
7. Clientul #2 este ultimul și nu are cui să trimită, așa că managerul întoarce ERR.
8. Clientul #0 trimite 2 clientului #1.
9. Clientul #1 are registrul 499. Dacă ar adăuga 2, ar depăși MAX_SIZE. Se întoarce ERR.
10. Clientul #0 trimite 1 clientului #1.
11. Clientul #1 are registrul 499. Se adaugă 1 cu succes.
12. Clientul #0 trimite 0 clientului #1.
13. Operația are succes.

Observații:

1. Transmiterea de numere prin send() nu este restricționată de valoarea din registrul expeditorului, ci de cea a destinatarului.

2. Atribuirea oricărei valori în propriul registru, `set_register()`, se poate desfășura oricând îi vine rândul clientului.

Fluxul de mesaje în rețea

Pentru că API-ul este uniform, managerul a socotit că ar fie bine să secvențializeze acțiunile clienților astfel:

1. Apelează `begin_message(msg)` din partea emițătorului
2. Ciclează pentru `MAX_CYCLES`
 1. Pentru fiecare client de la 0 la `nr_clients-1`
 1. Apelează `update()` din partea acelui client

Încă un exemplu ilustrat se găsește mai jos, pentru cazul în care emițătorul este clientul 0 și receptorul este clientul `n-1`, când valoarea de transmis este mică.

```
sequenceDiagram
    participant M as Manager
    participant A as Ana <br/> #35;0
    participant B as Bogdan <br/> #35;1
    participant C as Cristi <br/> #35;2
    autonumber

    note over A,C: Toate registrele sunt 0.
    note over M: msg = 30

    M->>A: begin_message(msg)

    activate A
    note right of A: set_register(msg)
    deactivate A

    note over M,C: Încep ciclul de update().

    A->>B: send(get_register())
    B->>A: OK

    activate B
    note right of B: registru(Bogdan) += 30
    deactivate B

    B->>C: send(get_register())
    C->>B: OK

    activate C
    note right of C: registru(Cristi) += 30
    deactivate C

    note over C: Determin că sunt ultimul client. <br/> Deci acționez write().
    C-->>C: write(get_register())
```

Starea fiecărui client

Din interiorul funcției `update()` numărul de identificare al clientului curent este inaccesibil. Ca să puteți totuși reține în ce stare se află fiecare client, aveți la dispoziție structura `state`. Fiecărui client `i` se va aloca o stare, pe care o poate accesa folosindu-se de parametru al funcției `update()`, numit `state`.

Intrare

Managerul citește două variabile: numărul de clienți și numărul pe care ultimul client vrea să îl transmită primului, separate pe câte un rând nou, în această ordine.

```
nr_clients
msg
```

Ieșire

Fiecare client dispune de posibilitatea scrierii de numere la ieșirea standard, apelând `write()`. Pentru fiecare apel de `write(x)`, managerul va scrie o linie de forma `<id> write: <x>`. Vom verifica dacă receptorul, și doar el, va scrie numai și numai `msg`.

Observație: Pentru implementarea oferită deja în `client.cpp`, cea în care primul client transmite un mesaj ultimului, **NU** se va acorda niciun punctaj.

Restricții și precizări

- `MAX_SIZE` este 500
- `MAX_CYCLES` este 200
- Numărul maxim de clienți este 20
- Fișierul `client.cpp` pe care îl încărcați va fi rulat alături de `manager.cpp` și `client.h`. Acestea din urmă nu se vor schimba.

Următoarele acțiuni sunt **interzise**, fiind penalizate cu pierderea punctajului pe problemă.

- Includerea în fișierul `client.cpp` a altor headere/biblioteci decât `client.h`
- Folosirea de variabile globale sau statice

Exemplu intrare

```
5 12239
```

Exemplu ieșire

```
stdout: > 0 write: 12239
```

Ieșiri considerate invalide:

```
> 0 write: 12239
```

```
> 0 write: 12239
```

```
> 0 write: 12239
```

```
> 1 write: 12239
```

```
> 1 write: 12239
```

Download sursă [aici](#).